GIDFINE PHASE 2: SECURE CONVEX MUTATIONS REPORT

Server-Side Loan Workflow Mutations, Authorization Guards, Redaction, and Audit Verification

Repository:	hitracesolutions/loan-engine	**Active Branch:**	development (protected)
Base Commit:	afb4323 (verified)	**Spec Reference:**	Gidfine_Dev_Guide.md v1.1
Auth Target:	Convex Auth (@convex-dev/auth)	**Execution Date:**	October 21, 2026

1. Executive Summary

Phase 2 delivers the core secure server-side mutations and queries for the Gidfine mini-scale loan lifecycle. Implemented strictly in frontend/convex/loans.ts, all functions operate through verified cryptographic token identities (ctx.auth.getUserIdentity()) and enforce the centralized state-machine engine without trusting client-supplied parameters.

Strict Safety Invariants Enforced:

- **submitApplication:** Resolves caller to verified borrower; validates positive integer pesewas and tenure; transitions DRAFT → SUBMITTED; records immutable audit event.
- **proposeTerms:** Restricted to agency_admin / loan_officer; validates lender eligibility within same tenant agency; preserves version history by superseding pending quotes; leaves borrower pricing uncalculated pending approved pricing formula.
- **approveOffer:** Requires agency_admin / underwriter; enforces non-bypass (lender acceptance cannot substitute for agency approval); updates application to AGENCY_APPROVED.
- **recordConsent:** Restricted to verified owning borrower; requires AGENCY_APPROVED status; requires explicit affirmative consent (affirmedConsent: true); generates tamper-evident SHA-256 fingerprint under Act 843; transitions to BORROWER_ACCEPTED.
- **getLenderProposal:** Restricted to assigned lender; enforces strict server-side redaction (borrower full name, phone, email, token, ID documents, and agency margin are completely stripped); returns pseudonymized case ID.
- **checkDisbursementReadiness:** Verifies approval and cryptographic consent integrity without initiating payment or activating schedules; confirms funding is NOT marked transferred.

2. Mutation & Query Specifications (frontend/convex/loans.ts)

Function	Allowed Role(s)	Preconditions & State Guard	Side Effects & Audit Action
submitApplication (Mutation)	borrower	Caller must have active borrower profile in agency. Amount > 0 integer pesewas, tenure > 0 integer months. DRAFT → SUBMITTED.	Inserts applications row. Writes APPLICATION_SUBMITTED audit log with amounts.
proposeTerms (Mutation)	agency_admin, loan_officer	Application & lender must belong to caller's agency. Lender must be active. Existing PENDING quotes marked SUPERSEDED. Version incremented.	Inserts lenderProposals row with bps rates. Updates application to LENDER_PENDING. Writes PROPOSAL_CREATED log.
decideLenderProposal (Mutation)	lender	Caller must be assigned lender for application. Proposal status must be PENDING. Transitions LENDER_PENDING → LENDER_ACCEPTED/DECLINED.	Patches proposal status (ACCEPTED/DECLINED). Patches application status. Writes LENDER_PROPOSAL_ACCEPTED/DECLINED log.
approveOffer (Mutation)	agency_admin, underwriter	Application must be LENDER_ACCEPTED; proposal must be ACCEPTED by lender. Non-bypass: transitions LENDER_ACCEPTED → AGENCY_APPROVED.	Patches application status to AGENCY_APPROVED. Writes AGENCY_OFFER_APPROVED log.
recordConsent (Mutation)	borrower	Caller must be owning borrower. Application must be AGENCY_APPROVED. affirmedConsent === true. Recomputes SHA-256 fingerprint.	Inserts consents row with fingerprint. Patches application to BORROWER_ACCEPTED. Writes BORROWER_CONSENT_RECORDED log.

getLenderProposal (Query)	lender	Caller must be assigned lender. Server-side redaction strips borrower PII and agency margin.	Returns pseudonymized case ID (APP-XXXXXX), filtered proposal terms, and sanitized application details.
checkDisbursementReadiness (Query)	agency_admin, loan_officer, cashier	Requires BORROWER_ACCEPTED status, active proposal, and valid matching consent snapshot with valid SHA-256 hash.	Returns readiness determination without initiating real transfer or activating repayment schedule.

3. Verification Suite & Actual Results

Test Domain / Suite	Target / Runner	Result	Details & Guarantees Verified
Phase 2 Mutation Tests	node --test tests/mutations.test.ts	**9/9 PASS**	Unauthenticated rejection, role authorization, pesewas validation, version preservation, tenant isolation, bypass prevention, affirmative consent, PII redaction, and disbursement lockout.
Phase 1 State Machine Tests	node --test tests/stateMachine.test.ts	**7/7 PASS**	20-status state machine guards, assigned lender enforcement, non-bypass rules, and SHA-256 snapshot hashing.
Combined Frontend Test Suite	node --test tests/*.test.ts	**16/16 PASS (442 ms)**	Full suite pass with 0 errors across 2 suites.
Convex TypeScript Checks	npx tsc --noEmit	**PASS (Exit 0)**	0 type errors across Convex schema, mutations, and helpers.
Frontend Production Build	npm run build	**PASS (Exit 0)**	Vite bundled in 9.13s (dist/assets/index-*.js: 343.97 kB).
Python Backend Regressions	pytest tests/ -q	**PASS (6 passed)**	6 passed in 2.57s. FastAPI comparison and admin endpoints intact.

4. Outstanding Business Decisions & Deployment Prerequisites

Unresolved Business Rules (Preserved without assumptions):

1. **Pricing & Margin Policy:** Formula for converting lenderBaseRateBps + agencyMarginBps into borrowerRateBps remains unfinalized. Stored as raw bps without applying calculation assumptions.
2. **Statutory Consent Legal Text:** The legal clauses for Act 843 data protection and disclosure compliance must be provided by legal counsel prior to production consent capture.
3. **Confirmed Disbursement Mechanism:** Payment rails (bank transfer / mobile money reconciliation) must be defined before implementing real disbursement mutations.

Deployment Prerequisites (for Matthias):

- npx convex dev or npx convex deploy will register the new tables and mutations in the cloud project.
- Configure CONVEX_AUTH_ISSUER_DOMAIN in Convex dashboard environment variables.